Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Dersin tarihi 11 Aralık 2020 Cuma. Ders Zoom üzerinden yapıldı.
  - Asıl anlatım yaklaşık 00:07:30'da başlar. Kaydın başı bekleme ve selamlaşmadır.
  - Ders üç bölümdür. Aralar yaklaşık 01:11-01:23 ve 02:10-02:21 arasıdır.
- Dönem planı duyurusu (ikinci vize, final, kalan konular).
- Yeni bölüm: Ağaçlar (trees)
  - Ağaç ile graf (graph) farkı, köklü (rooted) ve köksüz ağaç.
  - Ağaç terminolojisi: kök, ebeveyn (parent), çocuk (child), kardeş (sibling), ata (ancestor), torun (descendant), yaprak, uç düğüm (terminal vertex), ara düğüm (internal vertex), alt ağaç (subtree).
  - Seviye (level) ve yükseklik (height).
  - İkili ağaç (binary tree), tam ikili ağaç (full binary tree), dengeli ağaç.
  - İfade ağaçları ve işlem önceliği.
  - n düğümle çizilebilen farklı ikili ağaç sayısı.
  - İkili arama ağacı (binary search tree, BST): kelime örneği, sıralı diziden dengeli BST kurma.
  - İkili ağacın dizide tutulması: sol çocuk $2i$, sağ çocuk $2i + 1$.
- Ağaçta gezinme (traversal): önden sıralı (preorder), içten sıralı (inorder), sondan sıralı (postorder).
- Huffman kodlaması ve Huffman ağacı.
- Kapsayan ağaç (spanning tree)
  - Derin öncelikli arama (Depth First Search, DFS) ve yığın (stack).
  - Geniş öncelikli arama (Breadth First Search, BFS) ve kuyruk (queue).
  - En küçük kapsayan ağaç (minimum spanning tree) ve Prim algoritması.
  - Japonya şehirleri haritası üzerinde Prim algoritmasının görsel bir demosu.

Hocanın Özellikle Vurguladığı Kısımlar

- Her ağaç bir graftır, ama her graf bir ağaç değildir.
  - Ağaçta kapalı döngü yoktur. Bir düğümden başka bir düğüme tek bir yol vardır.
- Kök bilgisi verilmedikçe hiçbir düğüme "kök" diyemezsiniz.
  - Alışkanlık olarak en tepedeki düğüm kök kabul edilir. Ama bu ancak bariz bir durumda yapılır.
- "Ağaç" deyince yalnız ikili ağaç düşünmeyin. Bir düğümden istediği kadar dal çıkabilir.
- Dengeli BST kurarken her eklemeden sonra kökün solunda ve sağında aynı sayıda (en fazla bir fark) eleman kalmalı.
  - Bir alt ağaçta sol dal dolmadan sağ dala eleman konmaz. Bu kurala dikkat edin.
- Huffman algoritmasında en sık yapılan hatalar:
  - Toplama girmiş bir değeri tekrar toplamaya sokmak. Her değer yalnız bir kez toplanır.
  - Ağacı kurarken bir kuralı tutarlı uygulamamak. Kural: ilk okunan değer sola, ikincisi sağa.
- Huffman kodu kökten yaprağa (yukarıdan aşağı) okunur. Aşağıdan yukarı okunursa kod çözülemez.
- Prim algoritmasında üzeri çizilmiş (ziyaret edilmiş) düğümlerin sütununa bakılmaz. Yalnız ziyaret edilen düğümlerin satırlarında, henüz çizilmemiş sütunlardaki en küçük değer

aranır.
- Bir sonraki ders Kruskal algoritması ile devam edilecek.

Kısa Tekrar Notları

- Ağaç: döngü içermeyen bağlı graf. n düğümlü ağaçta $n-1$ kenar vardır.
- Seviye: düğümden köke olan kenar sayısı. Kökün seviyesi 0'dır.
- Yükseklik: seviyelerin en büyüğü, yani köke en uzak yaprağın uzaklığı.
- Ara düğüm: kendisinden dal çıkan düğüm (kök dahil). Uç düğüm (yaprak): dal çıkmayan düğüm.
- İkili ağaç: her düğümün en fazla 2 çocuğu vardır. İkili ağaçta l. seviyede en fazla 2^l düğüm olur.
- Tam ikili ağaç (full binary tree): uç düğümler dışındaki her düğümün tam 2 çocuğu vardır.
  - i ara düğüm varsa $i+1$ yaprak ve toplam $2i+1$ düğüm vardır.
  - t yapraklı ikili ağaçta $h \geq \lceil \log_2 t \rceil$.
- Dengeli ağaç: yüksekliği h olan ağaçta bütün yaprakların seviyesi h veya $h-1$'dir.
- n düğümlü farklı ikili ağaç sayısı Catalan sayısıdır: $\frac{1}{n+1}\binom{2n}{n}$. 1, 2, 3 düğüm için 1, 2, 5.
- BST: sol alt ağaçtaki her değer kökten küçük, sağdaki her değer kökten büyüktür.
- Diziyle ikili ağaç: kök indisi 1 ise i düğümünün sol çocuğu $2i$, sağ çocuğu $2i+1$.
- Gezinme kuralları ($T_1, T_2, \ldots, T_n$ soldan sağa alt ağaçlar):
  - Preorder: kök, sonra $T_1, T_2, \ldots$ (her biri preorder).
  - Inorder: T_1, sonra kök, sonra $T_2, \ldots$ (her biri inorder).
  - Postorder: $T_1, T_2, \ldots$, en sonda kök.
- Huffman adımları: frekansları küçükten büyüğe sırala; en küçük iki değeri topla; toplamı sırayı bozmadan yerleştir ve "ara düğüm" diye işaretle; n sembol için dizi $2n-1$ elemana ulaşınca dur; ağacı sondan başa kur; sol dala 0, sağ dala 1 yaz.
- DFS yığın, BFS kuyruk kullanır.
- Kapsayan ağaç: grafın bütün düğümlerini içeren ve döngüsü olmayan alt graf.
- En küçük kapsayan ağaç: kenar ağırlıkları toplamı en küçük olan kapsayan ağaç.

Detaylı Açıklamalar

Ağaç ve graf

Graf düğümlerden (vertex) ve kenarlardan (edge) oluşur. Grafta kapalı döngü olabilir. Döngü yoksa ve graf bağlıysa bu bir ağaçtır. Bu yüzden ağaçlar grafların özel bir alt modelidir.

Ağaçta iki düğüm arasında tek bir yol vardır. Grafta ise döngüler yüzünden aynı düğüme farklı yollardan gidilebilir. Bu fark, iki yapı için farklı algoritmaların kullanılmasının sebebidir.

Köksüz bir ağacın herhangi bir düğümü kök seçilebilir. Hoca bunu "ağacı kökten tutup üzüm salkımı gibi silkelemek" diye anlattı. Seçilen düğüm en üste gelir, diğerleri aşağı sarkar.

Terminoloji (aile ağacı benzetmesi)

- Ebeveyn (parent): kendisinden dal çıkan düğüm. Çocukları (child) bu dallardaki düğümlerdir.
- Kardeş (sibling): aynı ebeveynden çıkan düğümler.
- Ata (ancestor): bir düğümün kökle arasındaki yol üzerinde, üstte kalan bütün düğümler.
- Torun (descendant): bir düğümün altındaki bütün düğümler (çocukları dahil).
- Alt ağaç (subtree): bir düğüm ve onun bütün torunlarından oluşan ağaç.

İkili ağaç örneği

Slayttaki ağaçta kök A'dır. A'nın sol çocuğu B, sağ çocuğu C'dir. Ara düğümler A, C, D, E, H'dir. Uç düğümler B, F, G, I'dır. Ağacın yüksekliği 4'tür.

İfade ağaçlarında ara düğümler işleçlerdir, yapraklar işlenenlerdir. Örnek: $(A-B)\cdot C$ ifadesinde önce çıkarma, sonra çarpma yapılır. Ağacın yüksekliği iç içe işlem sayısına eşittir.

7 düğümlü üç farklı ikili ağacın yükseklikleri 2, 4 ve 6 olabilir. Düğüm sayısının aynı olması ağacın şeklini belirlemez.

Dengeli ikili arama ağacı kurma

Sıralı dizi: 1, 3, 4, 5, 6, 10, 16, 17, 19. Hoca elemanları sırayla ekledi. Her eklemeden sonra iki kuralı kontrol etti:

- Sol dal dolmadan sağ dala eleman konmaz.
- Kökün solunda ve sağında aynı sayıda eleman kalmalıdır (tek sayıda elemanda en fazla bir fark).

Kural bozulursa elemanlar sıralamayı koruyarak bir konum "yukarı itilir" (kök sağdaki elemanla yer değiştirir, diğerleri kayar). Slayttaki son ağaç:

Düğüm	Sol çocuk	Sağ çocuk
6 (kök)	4	16
4	3	5
3	1	-
16	10	19
19	17	-

Kökün iki tarafında 4'er eleman var. Arama örnekleri:

- 17 aranırsa: 6'dan büyük, 16'dan büyük, 19'dan küçük, bulundu.
- 2 aranırsa: 6'dan küçük, 4'ten küçük, 3'ten küçük, 1'e gelinir. 2 ağaçta yok.

Kelime örneği: "computers are an important technological tool" cümlesinin kelimeleri sözlük sırasına göre BST'ye yerleştirildi. "tennis" arandığında computers, important, technological, tool sırasıyla gezilir. tool'un sol çocuğu olmadığı için "tennis" ağaçta yoktur. Eklenecek olsaydı tool'un sol çocuğu olurdu.

İkili ağacın dizide tutulması

Kökün indisi 1 olsun. i indisli düğümün sol çocuğu $2i$, sağ çocuğu $2i+1$ indisine yazılır. Slayttaki ağaç (23, 13, 19, 47, 11, 43, 7, 29, 52) dizide şöyle durur:

İndis	1	2	3	4	5	6	7	8	12
Değer	23	13	47	11	19	43	52	7	29

43 aranırsa: indis 1'de 23, $43 > 23$, sağa git: $2 \cdot 1 + 1 = 3$. İndis 3'te 47, $43 < 47$, sola git: $2 \cdot 3 = 6$. İndis 6'da 43, bulundu.

50 eklenecekse: $50 > 23$, $50 > 47$, $50 < 52$. 52 indis 7'de olduğu için 50, $2 \cdot 7 = 14$ numaralı göze yazılır.

Ağaçta gezinme (traversal)

Slayttaki ağaç (Rosen'deki standart örnek):

- a'nın çocukları: b, c, d.
- b'nin çocukları: e, f. e'nin çocukları: j, k. k'nin çocukları: n, o, p.
- d'nin çocukları: g, h, i. g'nin çocukları: l, m.

Sonuçlar:

Gezinme	Sıra
Preorder	a, b, e, j, k, n, o, p, f, c, d, g, l, m, h, i
Inorder	j, e, n, k, o, p, b, f, a, c, l, g, m, d, h, i

Gezinme	Sıra
Postorder	j, n, o, p, k, e, f, b, c, l, m, g, h, i, d, a

Inorder'da bir düğümün ilk alt ağacı okunur, sonra düğümün kendisi, sonra kalan alt ağaçlar okunur. Kural her alt ağaçta tekrar uygulanır.

Huffman kodlaması

Neden gerekli? ASCII'de her karakter 8 bittir. Çok kullanılan karakter de az kullanılan karakter de aynı yeri kaplar. Sık kullanılan karaktere kısa kod, seyrek kullanılana uzun kod verilirse veri sıkıştırılır. JPEG ve RAR gibi sıkıştırma yöntemlerinin bazı adımları Huffman kodlamasını kullanır.

Örnek: a(25), e(14), k(8), l(7), m(4), z(2). Toplam 60 karakter.

Adımlar:

1. Sırala: 2, 4, 7, 8, 14, 25.
2. $2+4=6$, $6+7=13$, $8+13=21$, $14+21=35$, $25+35=60$. Her toplam ara düğümdür.
3. Kodda ara düğüm işareti için toplamın önüne eksi konur. Son dizi: 2, 4, -6, 7, 8, -13, 14, -21, 25, -35, -60. 6 sembol için $2 \cdot 6 - 1 = 11$ eleman.
4. Ağaç sondan kurulur: -60 kök, solunda -35, sağında 25. Eksi işaretli değerler yeniden açılır. Her açılışta ilk okunan sola, ikincisi sağa gelir.
5. Sol dala 0, sağ dala 1 yazılır.

Sembol	Frekans	Kod	Bit	Toplam bit
a	25	1	1	25
e	14	01	2	28
k	8	001	3	24
l	7	0000	4	28
m	4	00010	5	20
z	2	00011	5	10

Toplam 135 bit. 8 bitlik kodla $60 \cdot 8 = 480$ bit gerekirdi.

Neden yukarıdan aşağı okunur? Huffman kodu önek kodudur (prefix code): hiçbir kod başka bir kodun başlangıcı değildir. Bu yüzden sıkıştırılmış bit dizisi soldan okunarak tek biçimde çözülür. Örnek: `0011` dizisi "k" (001) ve "a" (1) olarak çözülür. Kodlar aşağıdan yukarı okunsaydı a = 1, e = 10, k = 100 olurdu. 1 hem a'nın kodu hem de e'nin ve k'nin başlangıcı olduğu için dizi çözülemezdi.

Derin öncelikli arama (DFS)

Kök düğümden başlanır. Gidilebilecek en uzak düğüme kadar gidilir. Gidecek yer kalmayınca geri dönülür ve ziyaret edilmemiş bir komşuya geçilir. Tüm düğümler gezilince bitirilir. Yığın kullanılır: düğüme gelince push, gidecek yer kalmayınca pop.

Slayttaki graf: A-B, A-H, A-D, D-F, B-E, B-C. A'dan başlanır.

Adım	İşlem	Yığın	Çıkan	Çıktı
1	Push(A)	A	-	A
2	Push(D)	D-A	-	D
3	Push(F)	F-D-A	-	F
4	Pop()	D-A	F	-
5	Pop()	A	D	-
6	Push(H)	H-A	-	H
7	Pop()	A	H	-
8	Push(B)	B-A	-	B

Adım	İşlem	Yığın	Çıkan	Çıktı
9	Push(E)	E-B-A	-	E
10	Pop()	B-A	E	-
11	Push(C)	C-B-A	-	C
12	Pop()	B-A	C	-
13	Pop()	A	B	-
14	Pop()	-	A	-

Çıktı: A, D, F, H, B, E, C.

Geniş öncelikli arama (BFS)

Düğümler seviye seviye gezilir. Hoca bunu suya atılan taşın oluşturduğu halkalara benzetti. Bir seviyedeki bütün düğümler bitmeden sonraki seviyeye geçilmez. Kuyruk (ilk giren ilk çıkar) kullanılır.

Slayttaki graf: A-B, A-C, B-F, C-E, C-D, D-H, H-J. A'dan başlanır.

Adım	Eklenen	Çıkan	Kuyruk	Çıktı
1	A	-	A	A
2	B-C	A	B-C	B-C
3	F	B	C-F	F
4	E-D	C	F-E-D	E-D
5	-	F	E-D	-
6	-	E	D	-
7	H	D	H	H
8	J	H	J	J
9	-	J	-	-

Çıktı: A, B, C, F, E, D, H, J. Bir seviyedeki komşuların hangi sırayla ekleneceği serbesttir. Örneğin önce C sonra B eklenirse çıktı değişir. Bu yüzden DFS ve BFS'nin tek bir doğru çıktısı yoktur.

Prim algoritması

Problem: köydeki evleri (düğümler) su deposuna (düğüm 1) en az boru ile bağlamak. Kenar ağırlıkları metre cinsinden uzaklıktır.

Komşuluk matrisi kurulur. Köşegen 0'dır. Bağlantı olmayan çiftlere çok büyük bir sayı (M) yazılır, böylece algoritma onları seçmez. Matris köşegene göre simetriktir.

	1	2	3	4	5	6	7
1	0	1100	100	300	600	1000	400
2	1100	0	M	M	M	M	900
3	100	M	0	M	500	M	M
4	300	M	M	0	M	700	M
5	600	M	500	M	0	200	M
6	1000	M	M	700	200	0	800
7	400	900	M	M	M	800	0

Adımlar (başlangıç düğüm 1):

1. 1'den en kısa kenar: 1-3 (100).
2. {1, 3} içinden en kısa: 1-4 (300).
3. {1, 3, 4} içinden en kısa: 1-7 (400).
4. {1, 3, 4, 7} içinden en kısa: 3-5 (500).

5. {1, 3, 4, 5, 7} içinden en kısa: 5-6 (200).
6. Kalan düğüm 2: en kısa bağlantı 7-2 (900).

Toplam: $100 + 300 + 400 + 500 + 200 + 900 = 2400$. Bundan daha küçük toplamlı bir kapsayan ağaç yoktur.

Her adımda ziyaret edilen bir düğüme bağlı ve henüz ziyaret edilmemiş bir düğüme giden en kısa kenar seçilir. Ziyaret edilmiş iki düğüm arasındaki kenar seçilmez, çünkü döngü oluşturur.

Ders sonrası notlar

- **Renk sayısı:** Hoca her renk kanalı için "7 bit" dedi ve 2^{21} renk hesapladı. Her kanal 0-255 arası, yani 8 bittir. Toplam 24 bit ve $2^{24} = 16\,777\,216$ renk vardır.
- **Tam ikili ağaç tanımı:** Derste bir yerde "her düğümden en fazla iki kenar çıkıyorsa full binary" dendi. Bu ikili ağacın tanımıdır. Tam ikili ağaçta uç düğümler dışındaki her düğümün **tam** iki çocuğu vardır. Hoca bunu sonra doğru söyledi.
- **Yükseklik ve log:** "t uç düğümlü tam ikili ağaçta yükseklik $\log_2 t$'ye eşittir" ifadesi yalnız bütün yaprakların aynı seviyede olduğu (mükemmel) ağaç için doğrudur. Genel kural $h \geq \lceil \log_2 t \rceil$'dir. Bu eşitsizlik yalnız tam ikili ağaçta değil, her ikili ağaçta geçerlidir.
- **Sıfır düğümlü ikili ağaç:** Hoca "0 düğümle ağaç çizilemez" dedi. Ağaç sayımında boş ağaç da bir ağaç sayılır. Bu yüzden Catalan dizisi 1, 1, 2, 5, 14 diye başlar. 1, 2 ve 3 düğüm için hocanın verdiği 1, 2, 5 sayıları doğrudur.
- **Ağaç tanımı:** Ağacın tanımında "döngü yok" yanında "bağlı" olma şartı da vardır. Bağlı olmayan döngüsüz graf ağaç değil, ormandır (forest).
- **DFS çıktısı:** Hoca slaytı özetlerken çıktıyı "A, D, F, H, B, C" diye okudu ve E'yi atladı. Slayttaki tabloya göre doğru çıktı A, D, F, H, B, E, C'dir.
- **BST kurma yöntemi:** Derste anlatılan "her eklemeden sonra elemanları yukarı itme" yöntemi standart BST eklemesi değildir. Standart eklemede yeni eleman yaprağa eklenir, ağaç dengesiz kalabilir. Sıralı bir diziden dengeli BST kurmanın kısa yolu: ortadaki elemanı kök yap, sol ve sağ yarılara aynı işlemi uygula.
- **Huffman eşit frekanslar:** Huffman ağacı tek değildir. Eşit toplamlar olduğunda farklı ağaçlar ve farklı kodlar çıkabilir. Ama toplam bit sayısı her zaman en küçük değerdir. Bu örnekte Python ile kurulan Huffman ağacı da 135 bit verdi.
- **Kontrol:** Huffman kod uzunlukları ve 135 bit, kodların önek kodu olması, Prim sonucu (2400), DFS ve BFS çıktıları ile dizi indisleri Python ile doğrulandı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.